TuneSwipe System Architecture

1. Scope of This Architecture Document

This document focuses only on the structure of the TuneSwipe system: the major software components, how data moves between them, the backend API layer, the database layout, authentication, and deployment responsibilities. It intentionally avoids repeating the full project motivation, development process, and detailed implementation discussion that belong in the separate project documentation.

TuneSwipe is a full-stack music discovery application. A user signs in, receives a deck of Spotify-based song cards, previews songs when available, and swipes or presses buttons to like or pass each track. The system stores liked and passed songs per user so that future recommendations can avoid songs the user has already judged.

2. High-Level System Architecture

The application is separated into three main layers. The React frontend is responsible for the user interface and user interactions. The Express backend is responsible for Spotify communication, filtering, and Firestore writes. Firebase provides authentication and persistent storage. This separation keeps Spotify credentials and database-admin operations away from the browser.

```
Browser / Vite React Frontend
   │
   │  HTTP requests with current Firebase UID
   ▼
Node.js / Express Backend on Render
   │        │
   │        ├── Spotify Web API: search tracks and obtain metadata
   │        └── Firebase Admin SDK: read/write Firestore data
   ▼
Cloud Firestore
   └── users/{uid}/likedSongs and users/{uid}/passedSongs
```

Layer	Technology	Architecture Role
Frontend	Vite React app hosted on Vercel	Displays authentication screens, card stack, play button, like/pass buttons, counters, and empty/loading states.
Backend	Node.js + Express hosted on Render	Exposes REST endpoints, hides Spotify credentials, queries Spotify, filters repeated songs, and writes to Firestore.
Authentication	Firebase Authentication	Creates and identifies users. The frontend uses the signed-in user UID when requesting and saving data.
Database	Cloud Firestore	Stores each user's liked and passed songs in separate subcollections using Spotify track IDs as document IDs.
External API	Spotify Web API	Provides track search results, album images, artist/album, and preview URLs when available.

3. Frontend Architecture

The frontend is a Vite React application. Its architecture is component-based: App.jsx controls global state and data loading, while smaller components focus on one visual responsibility. This makes the interface easier to modify because the card display, buttons, header, loading state, and authentication screen are separated.

```
App.jsx
├── AuthScreen.jsx              login / signup screen shown before authentication
├── Header.jsx                  app title and liked/passed counters
└── CardStack.jsx               current deck and swipe controls
    ├── SongCard.jsx            album image, title, artist, album, popularity, play button
    ├── ActionButtons.jsx       pass and like buttons
    ├── LoadingSpinner.jsx      loading state while songs are requested
    └── EmptyState.jsx          no-song / shuffle-again state
```

Component	Responsibility
App.jsx	Owns the current user, deck, liked count, passed count, loading state, and main fetch/save handlers.
AuthScreen.jsx	Handles login and signup using Firebase Authentication before the main app is shown.
CardStack.jsx	Displays the top cards, connects button clicks to the top card, and coordinates swipe behavior.
SongCard.jsx	Renders one track card and handles preview playback logic using available preview URLs.
ActionButtons.jsx	Provides the visible like and pass controls under the active card.
Header.jsx	Shows the app identity and the current user-specific liked and passed counts.

4. Backend Architecture

The backend acts as the secure middle layer between the browser and external services. The frontend never calls Spotify directly and never receives the Spotify client secret. Instead, the frontend calls Express routes. Express requests an access token from Spotify using the backend environment variables, searches for tracks, and returns only the simplified song fields needed by the interface.

Endpoint	Architectural Purpose
GET /	Health check confirming the backend is running.
GET /songs?q=...&uid=...	Searches Spotify, filters out liked/passed songs for that user, and returns a deck of available songs.
POST /like	Saves a liked track under users/{uid}/likedSongs/{spotifyId}.
POST /pass	Saves a passed track under users/{uid}/passedSongs/{spotifyId}.
GET /likedSongs/:uid	Returns the user's liked tracks for playlist display, counters, and recommendation context.

GET /passedSongs/:uid	Returns the user's passed tracks for counters or filtering checks.
GET /preview?song=...&artist=...	Attempts to locate a preview URL when the original Spotify search result does not include one.

5. Firestore Data Architecture

Firestore is organized around the authenticated user ID. Each user document can contain profile-level information, while song decisions are stored in subcollections. The most important architectural choice is using the Spotify track ID as the document ID. This naturally prevents the same song from being duplicated in the same liked or passed collection, and it allows the backend to filter recommendations by checking IDs.

```
users
└── {uid}
    ├── email: string
    ├── createdAt: timestamp
    ├── likedSongs
    │   └── {spotifyId}
    │       ├── spotifyId, title, artist, album, imageUrl, previewUrl
    │       └── likedAt: timestamp
    └── passedSongs
        └── {spotifyId}
            ├── spotifyId, title, artist, album, imageUrl, previewUrl
            └── passedAt: timestamp
```

Liked and passed songs use the same structure. Keeping these two subcollections consistent makes the backend filtering simpler: it can collect all saved Spotify IDs for the user, combine them into one exclusion set, and remove those tracks from new Spotify search results before returning the deck.

6. Main Runtime Data Flows

6.1 Login and App Start

```
User opens Vercel frontend
→ Firebase Auth checks session
→ If no user: show AuthScreen
→ If user exists: App.jsx stores user.uid
→ Frontend loads counts and requests songs for that uid
```

6.2 Song Recommendation Request

```
React App.jsx
→ GET /songs?q={query}&uid={uid}
→ Express obtains Spotify access token
→ Express searches Spotify tracks
→ Express reads user's likedSongs and passedSongs from Firestore
→ Express removes already-seen spotifyIds
→ Express returns available song objects to the frontend
```

6.3 Like or Pass Action

```
User swipes or clicks a button
→ Frontend calls POST /like or POST /pass with uid and song object
→ Express validates uid and spotifyId
```

```
→ Express writes the song to the matching Firestore subcollection
→ Frontend removes the card from the deck and updates the counter
```

7. Deployment Architecture

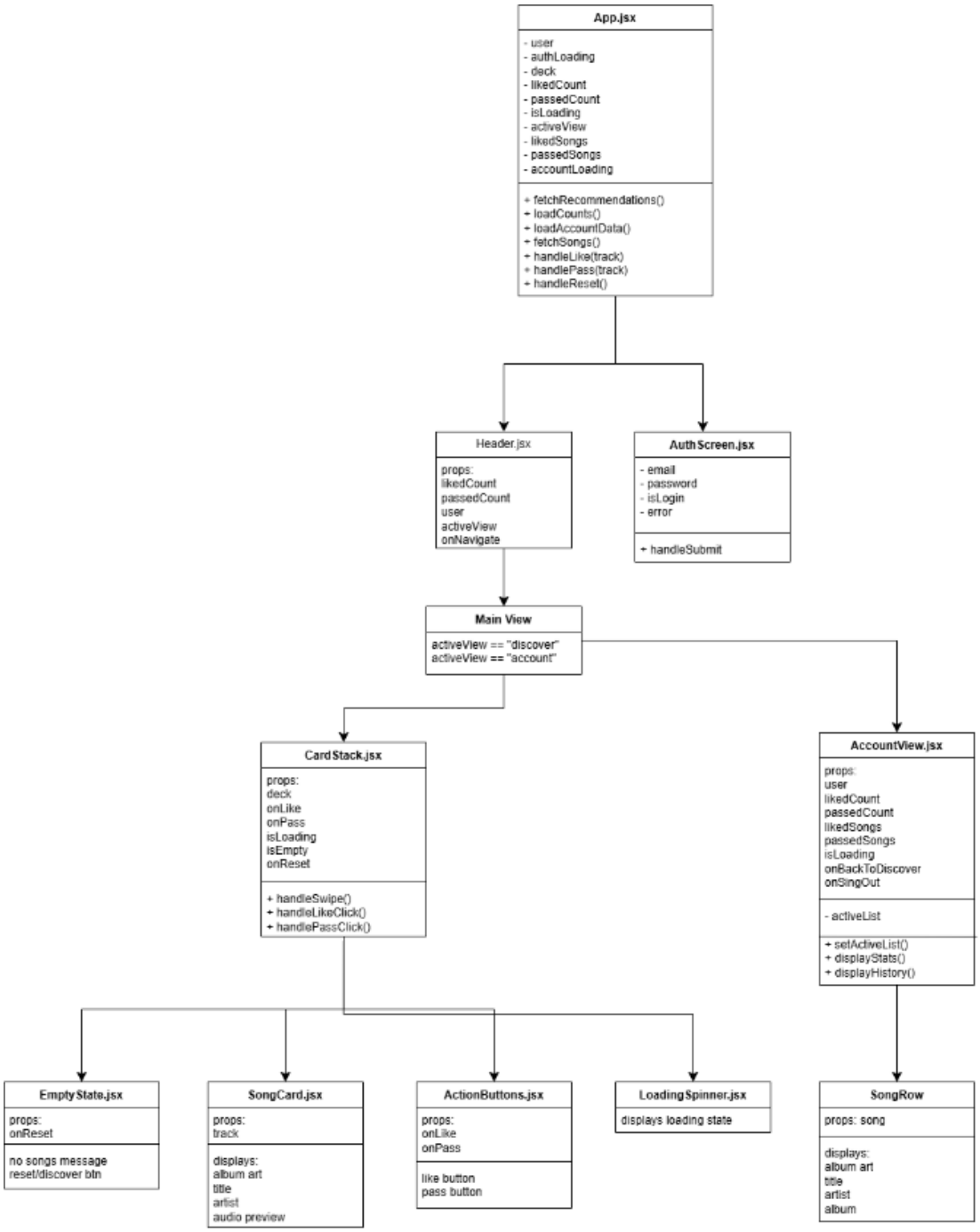
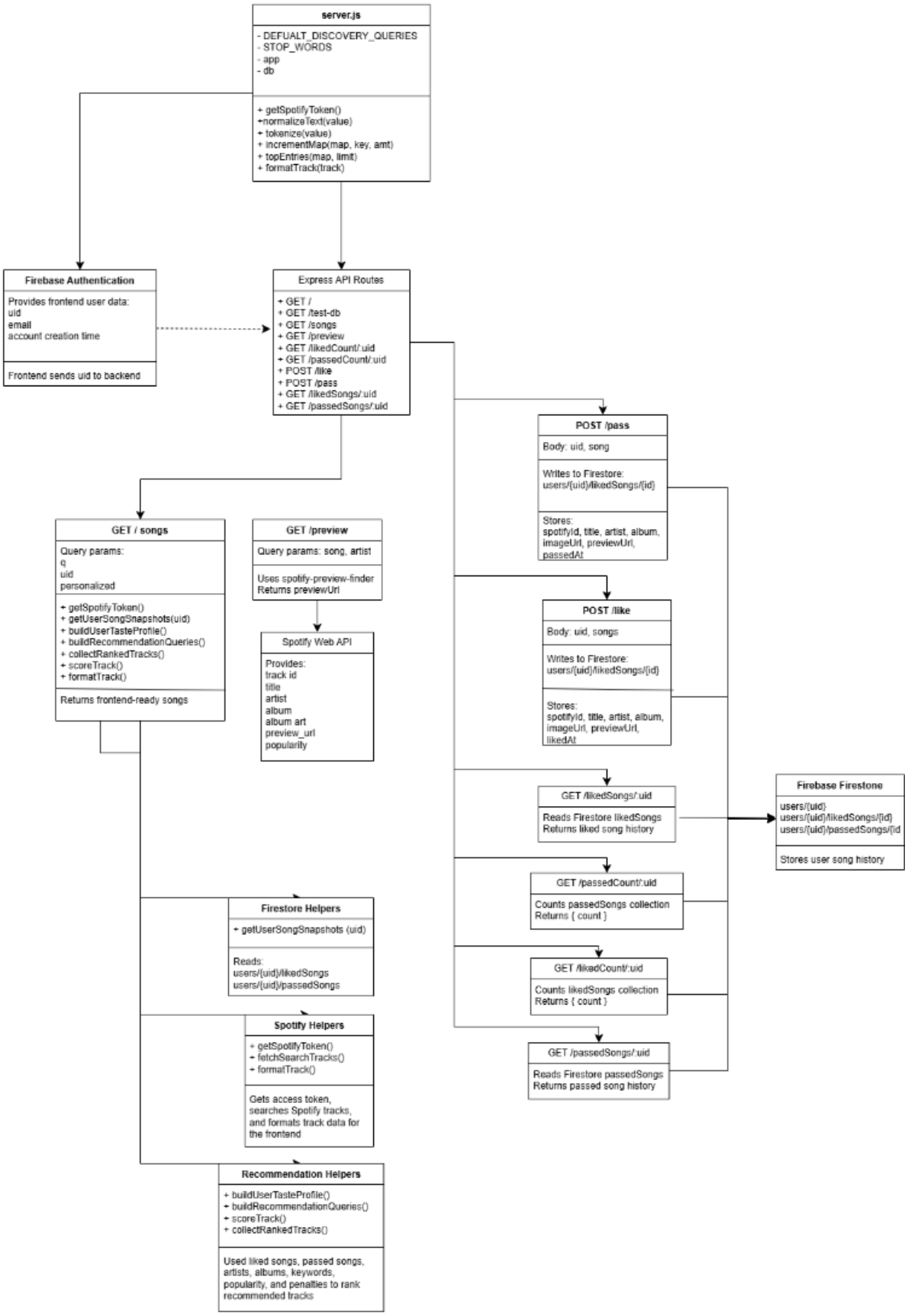
The deployed design uses separate hosting for the two application layers. The Vite React frontend is deployed to Vercel because it is a static client application after build. The Node/Express backend is deployed to Render because it must run a server process, store backend-only environment variables, and call Spotify and Firebase Admin services securely.

Service	Use in Architecture	Reason
Vercel	Frontend hosting	Builds the Vite app and serves the React client to users.
Render	Backend hosting	Runs server.js as a persistent Express service and exposes the API base URL.
Firebase Console	Auth + Firestore	Manages accounts and stores user-specific song data.
Environment variables	Configuration/security	Backend stores SPOTIFY_CLIENT_ID, SPOTIFY_CLIENT_SECRET, Firebase service credentials, and allowed frontend origin. Frontend stores only public Firebase web config and backend API URL.

8. Key Architecture Decisions

Decision	Architecture Benefit
Separate frontend and backend	Keeps the UI independent from server logic and prevents Spotify secrets from being exposed in browser code.
Firebase Authentication	Provides ready-made user accounts and stable UIDs without building a custom password system.
Firestore subcollections per user	Keeps liked and passed data isolated by user and makes user-specific filtering straightforward.
Spotify ID as Firestore document ID	Prevents repeated saves of the same track and supports efficient exclusion during recommendations.
Render for backend	Supports the Express server process required for Spotify token requests and database writes.
Vercel for frontend	Matches the Vite static build and separates public client hosting from private backend execution.

9. Frontend UML



10. Backend UML